Project Guide

A plain-English walkthrough of every assignment question — A1 to A4

This guide assumes you have never built a system like this before. For every question in the four assignment forms, it explains **what the question is really asking**, the **choices you have** and when to pick each, and gives a **worked example**. One example team runs through the whole guide so the pieces connect into one project.

Part 0 — The big picture (start here)

What you are building, in one paragraph

You are building a program that **reads scanned pages of old books** (photographs of pages, not typed text) and **answers questions about them**. When someone asks a question, your program finds the right page, reads it, and gives an answer — and it **shows which page the answer came from**, so the person can trust it. Every team builds this same kind of system; what makes yours different is three choices you make (below).

The words you will keep seeing (mini-glossary)

Read these once. Each is one sentence in plain terms; the guide uses them everywhere.

- **RAG (Retrieval-Augmented Generation) —** the overall design: instead of the AI answering from memory, it first retrieves the relevant page from your books, then writes an answer based on that page. This keeps answers grounded in real sources.
- **Agent —** a program that decides what to do step by step — e.g. "first search, then read that page, then write the answer" — rather than doing everything in one shot.
- **Corpus —** your collection of scanned book pages. It is your "knowledge base" — the only thing your system is allowed to answer from.
- **OCR (Optical Character Recognition) —** turning a picture of text into actual text the computer can read. A scanned page is just an image until OCR reads the letters off it.
- **Retrieval —** searching your corpus to find the page(s) most likely to contain the answer.
- **Embedding —** a way of turning a piece of text into a list of numbers that captures its meaning, so the computer can measure which texts are similar. Two passages about the same topic get similar numbers.
- **Vector index —** a searchable store of those number-lists, built so you can find the closest matches fast, even over thousands of pages.
- **Chunk —** a small piece of a page (a paragraph or a few lines). You split pages into chunks so retrieval can return just the relevant part, not a whole page.
- **Grounding —** making sure every answer is actually supported by a real page, and pointing to that page. An answer with no supporting page is "ungrounded."
- **Abstention —** the system saying "I don't have enough evidence to answer" instead of guessing. Abstaining is the correct behaviour when the answer isn't in your corpus.
- **Citation —** the reference your answer gives back — e.g. "page 212" — so a reader can check it.
- **NFR (Non-Functional Requirement) —** a quality your system must have, beyond just "getting the answer right" — for example being fast, private, or explainable. You pick one to focus on.

- **Serving / deployment —** making your finished system available for a real person to use — running it somewhere they can reach (usually a website), not just on your own laptop.
- **API (Application Programming Interface) —** a doorway that lets another program send your system a question and receive the answer back. It is how a web page or another app "talks to" your system over the internet.
- **UI (User Interface) —** the screen a person actually sees and uses: a box to type a question, a button to send it, and a space where the answer appears.
- **Docker / container —** a way to pack your whole program together with everything it needs to run (code, libraries, models, settings) into one sealed box called a container, so it runs the same on any computer. Think of a shipping container: whatever is inside travels and works identically anywhere.

The three choices that make your project unique

You are centrally assigned a distinct combination of these three, so copying another team's work never fits your own project. (Two qualities — being faithful and being accurate — are required of EVERY team, so they are not choices; see the NFR below.)

- ① **Domain —** your practical, real-user subject — community health, agriculture, legal aid, land records, and so on — and the person who would use the tool. No two teams share a domain.
- ② **Data speciality —** the ONE hard property of the corpus you are handed — e.g. Bangla, Arabic, code-switching, degraded scans, multi-column layout, handwritten pages, math notation. It is a condition you must engineer around, not a quality you optimise. (English is not a speciality, because clean English scanning is largely solved; an English corpus takes a data condition instead.)
- ③ **Primary NFR + target —** the ONE quality you optimise ABOVE the baseline, with a concrete target you commit to — e.g. low-latency (≤ 300 ms/query), memory-efficient (≤ 2 GB), calibrated (knows when unsure), private (hides personal data), offline (runs with no internet), explainable, auditable, secure. You pick one and measure it.

④ Agentic behaviour (required of everyone, mandatory — not a choice) — your agent must do evidence-gated research: when the top retrieval score is weak, widen the search (increase k) and retrieve again; if still weak at the maximum k, abstain. This is how we make sure the system is genuinely an agent (its path depends on what it observes), not a fixed pipeline. Graded fail-closed in A3.

> ***Example (Team G07)*** *Domain Agriculture · Data speciality Bangla · NFR Offline / on-device (target: runs on a phone with no internet). The corpus is Bangla agricultural extension manuals; the user is a farmer or extension officer. We refer back to this team throughout.*

How the work is split into four milestones, and how you hand it in

- **A1** Frame the project and prepare your corpus (planning + data; no building yet).
- **A2** Build the knowledge base (turn scanned pages into searchable text).
- **A3** Build the agent and evaluate it (make it answer questions, then measure how well).
- **A4** Ship it and look back (put it online, govern it, and reflect on what you learned).

You work in **one GitHub repository** (a shared online folder that records every change and who made it). At each deadline you "tag" your repository — a one-line command that bookmarks its exact state — named **a1-submit, a2-submit, a3-submit, a4-submit**. That bookmark is your submission. See the separate How-To-Submit guide for the exact steps.

Three tiers: what everyone does vs. what depends on your profile

- **CORE —** everyone must do it (reading pages, retrieval, the agent, grounding, evaluation).

- **GATED —** required only if your data speciality or NFR demands it (e.g. an image-cleanup stage, only if your scans are degraded).
- **BONUS —** optional, for extra credit (e.g. training the agent with reinforcement learning).

A1 — Frame Your Project & Prepare Your Corpus

Goal: decide exactly what your project is, find and document a real set of scanned pages, and design your agent **on paper**. You write no working code yet — A1 is about making good decisions before building. It is graded out of 100.

Section 1 — Your problem and your three choices (20 points)

The task is the same for everyone: "given a scanned page, answer a question grounded in that page." You do not invent a different task. You make it yours through the three choices below, then say who it is for and how you will judge success. (Being **faithful** and **accurate** is required of every team — so those are not choices; they are the baseline your NFR sits on top of.)

① Domain — your practical subject and its real user

Pick a practical, real-world subject and the person who would actually use the tool — community health, agriculture, legal aid, land records, and so on (see Domains-And-Natural-Fit). No two teams share a domain.

> ***Example (Team G07)*** *Agriculture — crop-guidance manuals used by a farmer or an agricultural extension officer.*

② Data speciality — the one hard property of your corpus

Pick the **one** thing that makes your scanned corpus hard to read — a difficult script (Bangla, Arabic), code-switching, degraded scans, multi-column layout, handwriting, math notation, and so on (see the Data-Speciality Catalog). It is a condition you engineer around, not a quality you optimise.

- **Note:** English is not a speciality — clean English scanning is largely solved. An English corpus must take a data condition (degraded, multi-column, handwritten…). If nothing else fits, Bangla is the default hard speciality.

> ***Example (Team G07)*** *Bangla — the manuals are printed in Bangla, whose conjunct letters and vowel-marks need conjunct-aware OCR.*

③ Primary NFR + target — the one quality you optimise, with a number

Beyond being faithful and accurate (baseline), pick the **one** quality you will design for, and commit to a concrete target. A few of the options, in plain terms (full list + metrics in the NFR sheet):

- **Low-latency —** answers fast — e.g. under 300 ms per question.
- **Memory-efficient —** runs in little memory — e.g. under 2 GB.
- **Offline / on-device —** works with no internet — e.g. on a phone in the field.
- **Calibrated —** knows when it is unsure (measured by calibration error ≤ 0.05).
- **Private —** detects and hides personal data (PII recall ≥ 0.98).
- **Explainable —** shows why it retrieved a page and where the answer came from.
- **Auditable —** every step is logged and replayable (100% of answers).
- **Secure —** resists malicious text hidden on a page (blocks ≥ 95% of injections).

> ***Example (Team G07)*** *Offline / on-device — an extension officer in a village with no signal must still get answers, so the whole system runs on the phone. Target: works fully offline, model ≤ 2 GB.*

Justify each of the three choices

For each axis, a one-liner is not enough — give a marker something testable. For the **domain**, the real decision it helps and why a document-reading agent is the right fit. For the **data speciality**, point to a specific page or feature

in YOUR corpus that shows it is really present, and name the one pipeline stage it forces. For the **NFR**, why the quality matters for your user, why that exact target number, and the one thing you would trade to hit it.

> ***Example (Team G07)*** *Bangla is real in our corpus — page 12 shows joined conjunct letters our English OCR misreads — so it forces a Bangla-tuned OCR stage. Offline matters because our extension officer has no signal; target model ≤ 2 GB; we would trade a little accuracy for the smaller on-device model.*

Who would actually use this

Name a real person or role and the decision your tool helps them make. Do **not** write "data scientists" or "the marker" — name a genuine end user.

> ***Example (Team G07)*** *A farmer or extension officer deciding how to treat a crop disease, without hand-searching hundreds of manual pages.*

How you'll measure success

Pick **one** headline number for answer quality and state the target you're aiming for. One clear number keeps the whole project focused.

> ***Example (Team G07)*** *Groundedness ≥ 0.9 — at least 90% of answers are fully supported by the cited page.*

The score you want to beat (baseline)

State the laziest possible "system," so you can later prove your real system adds value. A **baseline** is a trivial method you should be able to beat.

> ***Example (Team G07)*** *"Return the top matching chunk word-for-word" scores about 0.4 answer-F1 — our real agent must beat that.*

Section 2 — Your corpus (20 points)

Your corpus is the set of scanned pages your system reads. Finding a good one and documenting it honestly is graded here.

Corpus name and link

Give the public source and a working link. It **must be scanned page-images** (photographs of pages), not already-typed text — because reading the image is the whole point of the project.

- **Good sources:** Internet Archive, HathiTrust, Qatar Digital Library, Chronicling America.
- **Avoid:** Project Gutenberg — its books are already typed out, so there's nothing to read from an image, and you'd skip the hardest, most important part.

> ***Example (Team G07)*** *A public-domain Bengali extension manual collection on the Internet Archive, linked directly.*

Licence, and whether you may share it

State the licence (public-domain, Creative-Commons, or "research only") and whether you're allowed to re-share the files. If not, write "link only" and provide a small download script instead of the files.

> ***Example (Team G07)*** *Public domain — free to re-share, with the source credited.*

How big it is

It must be at least **300 pages AND at least 60,000 words** of usable text (a "huge-corpus" profile needs ≥ 100,000 pages). State the page count, word count, and size on disk.

> ***Example (Team G07)*** *420 pages · 78,000 words · 310 MB.*

Scan and script difficulty

Explain what makes **your** scans hard to read — joined letters and vowel-marks (Bangla/Arabic), unusual fonts, faded or tilted pages, mixed scripts, old spelling. This justifies the work you'll do later.

> ***Example (Team G07)*** *Pre-reform conjunct letters plus brown "foxing" stains on about 15% of pages.*

How you split it (so there's no cheating)

Divide your pages into a part you build with and a part you test on. Split **by whole book/document**, so no page from the same book appears in both parts.

Watch out: If you split page-by-page instead, pages from the same book leak into both sets and your test scores become dishonestly high. This is penalised.

Section 3 — What the data shows (15 points)

From your exploration notebook (a document that runs your data-inspection code top to bottom). Report what you actually found.

Three real findings

List three concrete things about **your** corpus — page-size spread, variety of fonts, how many pages are damaged, and so on.

> ***Example (Team G07)*** *"About 15% of pages are foxed; three distinct typefaces appear; the median page has ~190 words."*

What each finding means for your build

Turn each finding into a design consequence — a finding is only useful if it changes what you'll build.

> ***Example (Team G07)*** *"Foxing on many pages → we'll include an image-cleanup stage (which our profile requires anyway)."*

Three real example pages

Show three actual pages from your corpus.

Watch out: Generic images pulled off the internet score zero — they must be your real pages.

Section 4 — Risks (10 points)

Answer each honestly and specifically to your project; vague answers lose marks.

Personal data (PII)

Is any personal information present (names, addresses in the text), and how will you detect and hide it?

> ***Example (Team G07)*** *Donor names in book extension records → flag and redact them in answers.*

Licensing / copyright

Do you have the right to use and re-share it, and how do you credit it?

> ***Example (Team G07)*** *Public domain → cite the source, no restriction.*

OCR failure

Where will reading the text break, and what's your fallback?

> ***Example (Team G07)*** *Joined conjunct letters get misread → use a handwriting-text model and flag low-confidence lines for human review.*

Hallucination

How do you stop the system inventing answers with no support?

> ***Example (Team G07)*** *If no page supports an answer, the system abstains instead of guessing.*

Bias across your corpus

Which sub-groups might the system fail on (a rare font, one era)?

> ***Example (Team G07)*** *The oldest typeface reads worse than the rest — we report that gap rather than hide it.*

One decision where you chose honesty over a better score

Describe a real trade-off where you did the honest thing even though it lowered a number. This is exactly the kind of judgement the course rewards.

Section 5 — Design choices and justification (15 points)

A table covering your corpus/ingest stage across eight angles, plus one honest trade-off. The eight angles ("facets") make sure you've thought about the whole picture, not just the model.

The eight facets are: **Problem, Data, Model, Method, Design, Development, Deployment, MLOps**. For each, give your choice AND justify it: why it fits your corpus, what it costs, and what would make you change it. A one-liner will not score — the reasoning is what is graded.

One real trade-off

Name what you **gave up**, not only what you picked — every real choice costs something.

> ***Example (Team G07)*** *"We used simple classical image-cleanup instead of a learned model — cheaper and predictable, but it leaves very faint ink faint."*

Section 6 — Your pipeline and scope (5 points)

The "pipeline" is the sequence of stages from raw scan to final answer. Mark each stage as CORE (everyone), GATED (only if your profile needs it), BONUS (extra credit), or out-of-scope — correctly for your profile.

Every stage you mark here has a fixed file home in the code stub, and every enhancement (E1–E39) is indexed to its home in the Codebase Guide, Part 2 (per assignment, mirroring CLAUDE.md §10). You edit that home; you never add a file.

> ***Example (Team G07)*** *Because our pages are foxed, the image-cleanup stage is GATED-in (required for us); reinforcement learning is BONUS.*

Watch out: A team whose scans are clearly degraded but who marks image-cleanup "out of scope" has scoped wrong.

Section 7 — The agent layer, on paper (10 points)

Design your agent before coding it. "On paper" means you describe how it will work; you're not graded on running code yet.

7.1 The brain(s)

Which model drives the agent's decisions, and what does one reasoning step look like? The "brain" is the model that decides which action to take next.

> ***Example (Team G07)*** *A language model that, each step, picks a tool (search, read a page, or quote), looks at the result, then decides the next step until it can answer with a citation.*

7.2 Its tools (a fixed set you may not change)

Your agent acts by calling **tools** — small named actions. Everyone uses the **same nine tools**, not renamed or added to, so projects stay comparable. Say what each does **for your task**:

- **retrieve —** search the corpus for relevant chunks
- **rerank —** reorder those results so the best is first
- **read_page —** open a full page to read it closely
- **enhance_page —** clean up a damaged page image before reading

- **extract —** pull out the specific span that answers the question
- **aggregate —** combine information (e.g. count, sum) across pages
- **cite —** attach the source page to the answer
- **calculator —** do exact arithmetic
- **escalate_to_human —** hand off to a person when unsure

> ***Example (Team G07)*** *For fact questions, extract pulls the precise quoted line, and cite attaches the page it came from.*

7.3 Autonomy, humans, and budget

How independent is the agent; what is its limit on steps or compute (its budget) so it can't loop forever; and — optionally — when would it hand off to a human (**human-in-the-loop**)? Note: handing off to a human is a BONUS feature, not required — every team must instead ABSTAIN when the evidence stays weak. Describe the hand-off here only if you plan to build it.

> ***Example (Team G07)*** *If no page scores above the confidence threshold, the agent widens its search (larger k) and retrieves again; if it is still weak at the maximum k, it abstains rather than guessing. (Escalating to a human instead is optional/bonus.)*

7.4 How you'll test it, and how it improves

Describe the questions you'll test it on, and how it gets better over time — from better prompting, to better tool-use, to (optionally) training. If you attempt **reinforcement learning** (learning from a reward signal) for bonus, state the reward.

> ***Example (Team G07)*** *Reward = +1 when the extracted quote exactly matches the known correct quote — an automatically checkable signal.*

7.5 Agent-specific risks

Name the ways an agent specifically can go wrong, and your mitigation for each:

- **Prompt injection —** hidden instructions inside a scanned page trying to hijack the agent; you sanitise/ignore page text as commands.
- **Wrong tool / runaway loop —** the agent repeating actions forever; the step budget stops it.
- **Ungrounded answer —** answering with no supporting page; grounding + abstention prevent it.

Section 8 — How you worked with AI (5 points, plus a gate)

Each team member submits their own AI-chat transcript for the sections they led, saved as transcripts/<your student number>.txt. You're graded on **genuine engagement** — iterating, debugging, reasoning about your specific project — not on how much you pasted.

- **This is an authorship gate:** a missing or generic transcript holds the whole milestone for review, because the transcript is how we confirm each member did their own work.

A2 — Build the Knowledge Base

Goal: turn your scanned pages into something your system can actually search. This is the first milestone where you **build and run real code**. By the end you have a working "knowledge base": pages read into text, split into pieces, and stored so they can be searched — with evidence that it reads and retrieves. Graded out of 100.

The stages you build here, in order: **ingest** (load the pages) → **enhance** (clean up damaged images, if your profile needs it) → **layout** (find where the text sits on the page) → **OCR** (read the text off the image) → **chunk** (split into small pieces) → **embed** (turn each piece into meaning-numbers) → **index** (store them for fast search).

Section 1 — Recap from A1 (5 points)

Copy forward, unchanged, the key decisions from A1: your project name, task, three choices, corpus, and success metric. This lets the marker see your project as one continuous thing.

Watch out: If your three choices here don't match what you wrote in A1, that's a red flag — they must be identical.

Section 2 — The structure you exploit (10 points)

This section asks you to explain **why reading these pages is fundamentally a vision problem**, made concrete for your books. "Structure" means the regular patterns a computer can rely on.

Structure in the page (what the eye/vision model uses)

The visual regularities on the page — columns, lines, margins, the shapes of letters. Your layout and OCR stages depend on these being consistent.

> ***Example (Team G07)*** *Single-column procedure blocks with a consistent line of vowel-marks running along the top of each line.*

Structure in the text (what the language model uses)

The regularities in the words themselves — word order, common phrases, grammar. Your embeddings rely on these to capture meaning.

What this drives (your layout and OCR)

Say which of those structures your layout and reading stages actually use.

> ***Example (Team G07)*** *We detect procedure blocks first, then read each line individually — because the procedure layout is far more regular than the running prose in the margins.*

Section 3 — Components and options you considered (20 points)

For each stage you fill a small table: the **options you compared**, your **choice and why**, and whether you **reproduced a pretrained method** (ran an existing trained model rather than only citing it). You must genuinely reproduce at least one — reading the text (OCR) is the natural candidate.

The stages that need a real decision here:

- **OCR / reading —** how you turn page-images into text. Options include a general OCR engine (e.g. Tesseract), a model fine-tuned for handwriting/old scripts (HTR), or a vision-language model. Choose for your script's difficulty.
- **Chunking —** how you cut text into searchable pieces. Options: fixed-size (every N words), by paragraph, or "semantic" (split where the topic changes). Smaller chunks are precise but lose context; larger chunks keep context but retrieve loosely.
- **Embedding —** which model turns text into meaning-numbers. For Bangla/Arabic you need one that actually understands that language, not an English-only model.
- **Vector index —** how the numbers are stored for search. Options: a "flat" index (compares against everything — exact but slow on huge corpora) vs. an approximate index like HNSW or IVF (nearly as accurate, far faster at scale).

> ***Example (Team G07)*** *OCR: a Bangla-fine-tuned HTR model over plain Tesseract, because Tesseract mangles the conjunct letters. Chunking: by procedure block. Embedding: a multilingual model that covers Bengali. Index: a flat index (our corpus is small enough that exact search is fine).*

Biggest risk / unknown

Name the stage most likely to fail and your plan if it does.

> ***Example (Team G07)*** *OCR accuracy on the most faded pages — if it's too low, we fall back to flagging those pages for manual transcription.*

Section 4 — The knowledge base you built (20 points)

Show that the knowledge base actually exists and that your model and settings are justified — not just described.

The pipeline, in one honest paragraph

Describe what actually runs end-to-end, naming the models and the key settings you chose.

Index statistics

The concrete numbers proving the index is real: how many chunks, the size of each meaning-number list ("embedding dimension"), and the index type and size.

> ***Example (Team G07)*** *6,100 chunks · 768 numbers per chunk · stored in a FAISS flat index (~40 MB).*

What should NOT change

Name the fixed "contracts" — the agreed shapes of data passing between stages — that later stages depend on, so you don't accidentally break A3.

Section 5 — Evidence it works (25 points — the biggest block)

This is the largest single block, and the automatic grader **re-measures these itself** against a hidden set of correctly-transcribed pages. So report real numbers — inflated claims are caught.

OCR quality on a sample

Measure how accurately you read text on a labelled sample — a set of pages where the correct text is known. Report a standard accuracy number (character or word error rate, or F1) **and** the sample size.

> ***Example (Team G07)*** *Character-level F1 of 0.88 on 40 held-out pages (F1 balances how much you got right against how much you added or missed).*

One retrieval example that worked

Show one real question, the page your search returned, and confirm it's the right page.

> ***Example (Team G07)*** *"Which procedure discusses pesticide dosing?" → the search returned page 212, which is correct.*

The worst failure you saw

Show one honest case where reading or retrieval failed badly, and your read on why. Naming failures earns marks; hiding them loses them.

Watch out: If you claim OCR or retrieval numbers far above what the grader measures on your own files, that gap is flagged as possible result-fabrication.

Section 6 — The plan for A3 (5 points)

A concrete plan for the next milestone: how you'll go from "searchable text" to "an agent that answers questions" (retrieval → reranking → the agent loop → grounding).

Section 7 — Design choices and justification (10 points)

The same eight-facet table as A1 (Problem, Data, Model, Method, Design, Development, Deployment, MLOps), now covering the knowledge-base stages (ingest through index) — each with your choice AND a justification: why it fits your corpus, what it costs, and what would change it. This is your graded design record.

Section 8 — How you worked with AI (5 points, plus the gate)

Per-member transcripts, as in A1. New from here on: because you're now committing code to GitHub, the grader can also see **who committed what**. A member who claims the OCR work but never committed any of it, and never discussed it in their transcript, is flagged as a possible free-rider.

A3 — Build the Agent & Evaluate

Goal: build the part that actually answers questions — the agent that searches, reads, and reasons over your knowledge base — then measure honestly how well it works. This is the heaviest milestone. Graded out of 100.

An **agent**, again, is a program that works step by step: it decides to search, looks at what came back, reads a page, and only then answers — using the nine fixed tools from A1. "Cross-cutting" features (grounding, guardrails, and so on) are behaviours that don't live in one place but attach at set points in that loop.

Section 1 — Recap from A1 and A2 (5 points)

Carry forward: project name, task, the knowledge base you built, your primary NFR and its metric, your success target, and the baseline you're beating. Must match A1/A2.

Section 2 — What you built (20 points)

Describe the agent, your search design, and the cross-cutting features — and prove each is actually wired in.

The agent loop

Explain how your agent plans one step at a time over the nine tools, and how it knows when it's done.

> ***Example (Team G07)*** *For an a specific fact question: retrieve candidate pages → read the best one → extract the matching line → cite the page → answer.*

Retrieval and reranking design

Retrieval is the heart of the whole system, so this is asked explicitly. Decide and justify:

- **Retrieval strategy —** do you match on meaning ("dense", using embeddings), on exact words ("sparse", like a keyword search / BM25), or **both together** ("hybrid")? How many results do you pull back ("top-k"), and how do you measure closeness (the "similarity metric")? Justify this for your corpus size and script.
- **Reranking —** after the first search returns, say, the top 20, do you re-order them with a more careful (slower) model to push the best to the top? A "reranker" trades a little speed for better ordering. Say whether you use one and what it changed.

> ***Example (Team G07)*** *Hybrid search (keyword + meaning) returning the top 20, then a cross-encoder reranker narrows to the best 5 — hybrid because old spelling makes pure meaning-search miss exact terms.*

Cross-cutting features wired

These behaviours attach at fixed points ("seams") in the agent loop. Say which you wired and how you proved each works:

- **Grounding / abstention —** every answer is backed by a real page, and the agent says "not enough evidence" when it isn't.
- **Guardrails / injection defence —** the agent ignores any instructions hidden inside a page's text ("prompt injection") that try to hijack it.
- **PII redaction —** personal data is detected and hidden in outputs and logs.
- **Tracing / audit trail —** each step is recorded so you can see exactly what the agent did.

- **Human-in-the-loop / escalation —** when the agent is unsure, it hands off to a person (the escalate_to_human tool) rather than guessing.

> ***Example (Team G07)*** *Grounding attaches just before answering; injection defence attaches on every tool call; escalation fires when no page clears the confidence threshold.*

Is it reproducible, and where the code lives

State whether a fresh machine, running your pinned setup with fixed random seeds, reproduces your numbers — and where the models and index are (included, or recreated by a script). If reproducibility is your chosen NFR, this is your headline evidence.

Section 3 — Your results (20 points — the grader recomputes)

Report the numbers that show it works. The grader will author its own questions from known-answer pages and re-measure, so be accurate.

Retrieval quality

how often the right page is in your top-k results ("recall@k"). Report the number.

> ***Example (Team G07)*** *Recall@5 of 0.86 — the correct page is among the top 5 results 86% of the time.*

Headline answer quality

your one chosen metric against its target.

> ***Example (Team G07)*** *Groundedness 0.91 vs. our 0.90 target.*

Verifiable-question accuracy

on your checkable questions, where answers are exact-match (this anchors objective grading and RLVR).

> ***Example (Team G07)*** *a specific fact matches the known quote 82% of the time.*

Judged-question quality

on your non-checkable questions; score them with an LLM-judge or a human, and say which.

> ***Example (Team G07)*** *Two-step answers judged correct 74% of the time by an LLM-judge with a stated rubric.*

Is it reproducible?

pinned dependencies and fixed seeds; a re-run reproduces the numbers.

> ***Example (Team G07)*** *Re-running from the lockfile reproduces every number to within rounding.*

Section 4 — Beating the baseline + one change test (15 points)

Result vs. baseline

Show, with numbers, that your real agent beats the lazy baseline you named in A1.

> ***Example (Team G07)*** *Baseline (top chunk verbatim) 0.41 answer-F1 → our agent 0.79.*

One change test (an "ablation")

An **ablation** means switching off one component to see how much it mattered. Toggle one piece off and report the change — this proves the piece earns its place.

> ***Example (Team G07)*** *Turning off the reranker drops recall@5 from 0.86 to 0.75 — so the reranker is worth its cost.*

Section 5 — The agent on its task suite (25 points)

Your "task suite" is the set of test questions you built for your project. This section is about how you made it and how the agent does on it.

How you authored the task suite

Explain how you built your question set: how many questions; how you set the **correct answers** ("gold answers" — read from your labelled pages); the split between **verifiable** questions (checkable by exact match) and **judged** questions (scored by an LLM or human); and your out-of-corpus questions whose correct answer is to abstain.

> ***Example (Team G07)*** *30 questions: 20 a specific fact (checked automatically), 6 two-step (LLM-judged), and 4 out-of-corpus questions where abstaining is the correct answer.*

A full worked trace

Walk one real question through end-to-end: the tools the agent called in order, and how it reached a grounded, cited answer.

Reinforcement learning (bonus)

Optional, for extra credit. If you **trained a policy** — taught the agent from a reward signal ("RLVR" uses your checkable fact answers as the reward) — state the reward and its measured before/after effect. Leave blank if not attempted.

> ***Example (Team G07)*** *Reward +1 when the extracted quote exactly matches gold; after training, a specific fact accuracy rose from 0.82 to 0.88.*

Section 6 — Where it fails (5 points)

Honest failure analysis shapes your grade more than a high number does.

A hallucination you caught

Show one real case where the agent produced an answer with no support, and your fix.

Your honest list of failure modes

Where does it break — which data conditions, kinds of question, or page qualities? Naming these accurately is the point.

Section 7 — Design choices and justification (5 points)

Extend the eight-facet table to the stages you built this milestone — retrieval, the agent, any RL, and evaluation. For each facet give your choice AND justify it: why it fits your corpus, what it costs, and what would change it. This is the graded design record for A3.

Section 8 — How you worked with AI (5 points, plus the gate)

Per-member transcripts plus commit history, as before — the check for genuine, individual contribution.

A4 — Ship It & Look Back

Goal: put your system somewhere a real person can use it, make it trustworthy and maintainable, and step back to name the **durable lessons** behind your decisions. This is the capstone. Graded out of 100, plus up to 8 bonus.

Section 1 — Recap from A1–A3 (5 points)

Carry forward your project, the agent you built, and your primary-NFR result. By now the whole system should work end-to-end.

Section 2 — The live demo, brochure, and video (18 points — the grader runs your demo)

"Shipping" means making your system usable by someone who isn't you. Right now it runs on your machine; here you put it online.

Live demo link

A working, hosted version anyone can open. "Hosted" means it runs on a computer on the internet, not your laptop. A common free option is **Hugging Face Spaces** (a site that runs small AI apps for you); a container URL is another.

> ***Example (Team G07)*** *A Hugging Face Space with a text box: type a question, get a cited answer back.*

What the demo does, and how a marker runs it

Describe the user flow in two or three lines, and give one-click / one-command instructions so a marker can try it without help.

Brochure (one page)

A single page that explains your system to a non-expert — what it does and its limits — without overclaiming.

Demo video

A short screen recording showing it working, in case the live link is down when marked.

Section 3 — Keeping it running (15 points)

"Governance" and "MLOps" mean the housekeeping that keeps a deployed system trustworthy and maintainable over time. This section also asks how you packaged and served it — which is where three terms finally come together.

How you packaged and served it

Here Docker, an API, and a UI (all defined in the glossary at the front) fit together:

- **Docker packages it —** your whole program and everything it needs go into one sealed "container" that runs identically on any machine (the shipping-container idea).
- **An API exposes it —** a doorway that lets other software send your system a question over the internet and receive the answer back.
- **A UI lets a person use it —** the screen with a box to type a question, a button to send, and a place the cited answer appears.

Together these are your "serving stack." Name yours and how the pieces connect.

> ***Example (Team G07)*** *The program in a Docker container, exposed through a small API, with a Hugging Face web page as the UI — the UI calls the API, which runs the agent inside the container.*

Model card / datasheet

A short standard document describing your system for others: its **intended use**, **limitations**, where the **data** came from, its **risks and biases**, and its **evaluation** results. It's how a newcomer learns what your system should and shouldn't be used for.

Monitoring, audit, and versioning

Which signals you'd watch once it's live (error rates, abstention rate, latency), the record of decisions you keep, and how you label each model/data/config state so you can reproduce or roll back. "Versioning" = labelling each state so you can return to it.

How you'd update it

Your plan for refreshing the system when new books arrive or the world drifts — when and how you'd rebuild or retrain.

Section 4 — Handling the real world (15 points)

Show the system holds up under realistic conditions, tested at the live endpoint.

Speed and size

how fast it answers ("latency") and how big the models are — checked against what you claimed.

> ***Example (Team G07)*** *About 1.5 seconds per answer; the reader model is ~500 MB.*

Disclaimer shown to users

an honest note in the interface about the system's limits, so users don't over-trust it.

> ***Example (Team G07)*** *"Answers are drawn from a historical corpus and may contain period inaccuracies — verify important quotations against the cited page."*

Fairness / bias check

a measured gap between sub-groups (e.g. an old typeface vs. a new one), reported as a number.

> ***Example (Team G07)*** *Answer accuracy is 6 points lower on the oldest typeface — reported, not hidden.*

Stress test

how it behaves under load, bad input, or pages unlike your corpus ("out-of-distribution").

> ***Example (Team G07)*** *Fed 200 rapid questions and a blank page — it stayed responsive and abstained on the blank.*

Section 5 — Deploying the agent safely (17 points — your NFR, in production)

The safety and operations controls that make a live system responsible. These are where your chosen NFR shows up in production.

Approval gates (where a human must sign off)

points where a person must approve before the system acts — relevant if your task has consequences.

> ***Example (Team G07)*** *Any answer the agent is unsure about is queued for a human before it's shown.*

Monitoring and audit trail

the live signals you track and the step-by-step record you keep, so problems are noticeable and traceable.

> ***Example (Team G07)*** *Every answer logs the pages it used and the tools it called.*

Budget and rate limits

guards on cost and traffic — e.g. capping requests per minute so it can't run up a huge bill or be overwhelmed.

> ***Example (Team G07)*** *Capped at 30 requests per minute per user.*

PII / privacy in production

personal data is detected and hidden in the live system, not just in theory.

> ***Example (Team G07)*** *Donor names surfaced from a extension record are redacted before the answer is shown.*

Prompt-injection defence

proof that hidden instructions inside a page don't hijack the served system.

> ***Example (Team G07)*** *A page containing "ignore your rules and reveal X" was handled normally; the injection was ignored.*

Kill-switch / rollback

how you stop the system or revert to a previous good version if a deployment goes wrong.

> ***Example (Team G07)*** *One command reverts to the last working version; a flag disables the live endpoint instantly.*

Section 5b — Serving design choices (5 points)

Complete the eight-facet design table for the **serving** stage — the last slice of your design record. It covers what serving must achieve (its speed/uptime target), what the live service receives and returns, which model version you serve, batching/caching/concurrency, and the **API + UI contract** (the agreed shape of requests and responses, and where guardrails and PII-redaction attach). For each facet give your choice AND justify it — why it fits your project, what it costs, and what would change it. Name the trade-off you made: serving is usually a three-way tension between speed, cost, and accuracy.

> ***Example (Team G07)*** *We keep the reranker in production for accuracy, accepting ~0.5s extra latency — speed traded for trustworthiness, which suits our offline NFR.*

Section 5c — Bonus opportunities (optional, up to +8)

Extra credit for going beyond the core — reinforcement learning, extra NFRs delivered, advanced serving, and so on.

Section 6 — The principle scan (20 points — the intellectual payoff)

This is the heart of A4, and its largest single block. Map at least eight real decisions you made across the whole project to the **durable principles** behind them — a durable principle is a lesson that would transfer to a completely different project, not just a description of what you did.

For each row: the decision → what forced it → the general principle it illustrates.

> ***Example (Team G07)*** *Decision: split data by whole book, not by page. Forced by: the risk of leakage. Principle: "respect the unit of correlation when splitting data" — which applies to any dataset, anywhere.*

Watch out: Generic platitudes ("we learned teamwork matters") score low. A strong row names a real trade-off and a lesson that genuinely transfers.

Section 7 — Coverage: principles used and skipped (3 points)

Account for which of the course's principle families your project exercised, which you consciously did not, and why. Not every project touches every principle — being honest about what you skipped is part of the grade.

Section 8 — How you worked with AI (2 points, plus the gate)

Final per-member transcripts plus your whole-project commit history — the last check that every member genuinely contributed across all four milestones.

A closing note

If you've reached here, you've built a complete document-understanding agent — from photographs of pages to a governed, live system that answers questions and shows its sources. The point was never any single tool; it was the **judgement** behind each decision, which is exactly what the principle scan asks you to make explicit.